

Network IDS

Team:

Intern id: 322 Honey Jain,242 Vedshree Kanitkar,263 Jhanvi Singh

1. Methodology

1.1 Tools Used

- **Kali Linux** for traffic generation and testing.
- **Scapy** for packet parsing and IDS logic.
- **tcpdump** for capturing traffic into .pcap files.
- **Nmap** for generating scanning traffic.
- **pytest** for unit testing detection functions.

1.2 Detection Logic Implemented

1. ICMP Detection

- Flags echo requests/replies.
- Detects floods (repeated ICMP within short time).

2. TCP SYN Detection

- Flags SYN packets as possible connection attempts.
- Tracks unique destination ports to detect scans.

3. Scan Pattern Detection

- Detects when SYN packets are sent to many unique ports (SYN scans).
- Extended to detect NULL scans (-sN) and FIN scans (-sF).

4. Suspicious Behavior

- Threshold-based alerts for floods (e.g., >10 SYNs in <5 seconds).

2. Experiment Setup

Step 1 – Normal Traffic

- Ran `ping -c 4 127.0.0.1`.
- Captured traffic into `test.pcap` using `tcpdump`.
- Ran IDS against `test.pcap`.

```
(kali@kali) ~$ ls -lh test_scans.pcap
-rw-r--r-- 1 tcpdump tcpdump 36K Sep  2 11:28 test_scans.pcap

(kali@kali) ~$ ping -c 4 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.152 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.054 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.061 ms

— 127.0.0.1 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3370ms
rtt min/avg/max/mdev = 0.044/0.077/0.152/0.043 ms
```

Step 2 – Malicious Traffic

- Ran nmap -sS 127.0.0.1 (SYN scan).
- Ran nmap -sN 127.0.0.1 (NULL scan).
- Ran nmap -sF 127.0.0.1 (FIN scan).
- Captured traffic into test_scans.pcap.
- Ran IDS against test_scans.pcap.

```
GNU nano 8.3 offline_ids.py
from scapy.all import rdpcap, IP, TCP, ICMP
from collections import defaultdict, Counter

def analyze_pcap(pcap_file):
    packets = rdpcap(pcap_file)

    syn_tracker = defaultdict(set)  # Track SYN's + (src_ip: set(dest_ports))
    icmp_counter = Counter()        # Count ICMP packets per src
    tcp_flag_tracker = defaultdict(list)  # Track TCP flags per src

    for pkt in packets:
        if pkt.haslayer(ICMP):
            src = pkt[IP].src
            dst = pkt[IP].dst
            icmp_counter[src] += 1
            print(f"[ALERT] ICMP Packet: {src} -> {dst}")

        if pkt.haslayer(TCP):
            src = pkt[IP].src
            dst = pkt[IP].dst
            dport = pkt[TCP].dport
            flags = pkt[TCP].flags

            # SYN detection
            if flags == "S":
                syn_tracker[src].add(dport)
                print(f"[ALERT] TCP SYN: {src} -> {dst}:{dport}")

            # NULL scan (no flags set)
            elif flags == 0:
                tcp_flag_tracker[src].append("NULL")
                print(f"[ALERT] TCP NULL Scan: {src} -> {dst}:{dport}")

            # FIN scan
            elif flags == "F":
                tcp_flag_tracker[src].append("FIN")
                print(f"[ALERT] TCP FIN Scan: {src} -> {dst}:{dport}")

    # Detect Port Scans
    for src, ports in syn_tracker.items():
        if len(ports) > 10:  # threshold
            print(f"[SCAN DETECTED] {src} sent SYN's to {len(ports)} unique ports")

    # Detect ICMP Floods
    for src, count in icmp_counter.items():
        if count > 20:  # threshold, tune as needed
            print(f"[FLOOD DETECTED] {src} sent {count} ICMP packets")
```

```

# Detect ICMP Floods
for src, count in icmp_counter.items():
    if count > 20: # threshold, tune as needed
        print(f"[FLOOD DETECTED] {src} sent {count} ICMP packets")

# Detect repeated NULL/FIN scans
for src, scans in tcp_flag_tracker.items():
    if scans.count("NULL") > 5:
        print(f"[SCAN DETECTED] {src} performed {scans.count('NULL')} NULL scans")
    if scans.count("FIN") > 5:
        print(f"[SCAN DETECTED] {src} performed {scans.count('FIN')} FIN scans")

if __name__ == "__main__":
    analyze_pcap("test.pcap")

```

```

(myenv)-(kali@kali)-[~]
$ sudo python3 offline_ids.py

```

```

[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1 packet loss, time 3368ms
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1 65/0.047 ms
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1 at 2025-09-02 11:28 EDT
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1 (1)
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1 (127.0.0.1) are in ignored state
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1 (1)
[ALERT] ICMP Packet: 127.0.0.1 → 127.0.0.1
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:21 banned in 0.15 seconds
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1723 at 2025-09-02 11:28 EDT
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1025 (1)
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:3389
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:22 (127.0.0.1) are in ignored state
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:995 (1)
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:256
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:199 banned in 0.16 seconds
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:3306 at 2025-09-02 11:28 EDT
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:993 (1)
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:8888 (1)
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:135
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:443 (127.0.0.1) are in ignored state
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:445 (1)
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:25
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:53 banned in 0.17 seconds
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:587
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:23
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:554

```

```

[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:9415
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:8002
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:137
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1093
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:10616
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1301
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1062
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:6666
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1322
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:8100
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:18040
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:100
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:888
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:3880
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1971
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:7103
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1175
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:2010
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1040
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1010
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:5060
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1050
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1287
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1073
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:4125
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:5190
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1201
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:2048
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:4443
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:55056
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:6789
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:6100
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:5357
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:55600
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1217
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1048
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:9091
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:82
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:7512
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:6792
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:1035
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:726
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:6881
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:15002
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:222
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:18101
[ALERT] TCP SYN: 127.0.0.1 → 127.0.0.1:2920
[SCAN DETECTED] 127.0.0.1 sent SYNs to 96 unique ports in 0.15 seconds

```

```

(kali@kali)-[~]
$ ping -c 4 127.0.0.1
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data:
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.051 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.048 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.165 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.098 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3368ms
rtt min/avg/max/mdev = 0.048/0.090/0.165/0.047 ms
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-02 11:28 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000011s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.15 seconds
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-02 11:28 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000010s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.16 seconds
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-02 11:28 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000011s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

```

Step 3 – Raw Packet Verification

- Verified contents with:

`tcpdump -r test_scans.pcap | head`

```
(myenv)-(kali@kali)-[~]
└─$ tcpdump -r test_scans.pcap | head
reading from file test_scans.pcap, link-type EN10MB (Ethernet), snapshot length 262144
11:28:26.296966 IP localhost > localhost: ICMP echo request, id 5, seq 1, length 64
11:28:26.296980 IP localhost > localhost: ICMP echo reply, id 5, seq 1, length 64
11:28:27.313229 IP localhost > localhost: ICMP echo request, id 5, seq 2, length 64
11:28:27.313240 IP localhost > localhost: ICMP echo reply, id 5, seq 2, length 64
11:28:28.336585 IP localhost > localhost: ICMP echo request, id 5, seq 3, length 64
11:28:28.336609 IP localhost > localhost: ICMP echo reply, id 5, seq 3, length 64
11:28:29.361248 IP localhost > localhost: ICMP echo request, id 5, seq 4, length 64
11:28:29.361273 IP localhost > localhost: ICMP echo reply, id 5, seq 4, length 64
11:28:29.489384 IP localhost.56578 > localhost.smx: Flags [S], seq 4132458591, win 1024, options [mss 1460], length 0
11:28:29.489405 IP localhost.smx > localhost.56578: Flags [R.], seq 0, ack 4132458592, win 0, length 0
tcpdump: Unable to write output: Broken pipe
```

Step 4 – Unit Testing

- Wrote test_ids.py with functions:

- test_detect_icmp()
- test_detect_syn()
- test_detect_null()
- test_detect_fin()

- Ran with `pytest -v`.

```
GNU nano 8.3 /home/kali/.ssh/authorized_keys
ids_core.py
from scapy.all import IP, TCP, ICMP

def detect_icmp(pkt):
    if pkt.haslayer(ICMP):
        return f"[ALERT] ICMP Packet: {pkt[IP].src} → {pkt[IP].dst}"
    return None

def detect_syn(pkt):
    if pkt.haslayer(TCP) and pkt[TCP].flags == "S":
        return f"[ALERT] TCP SYN: {pkt[IP].src} → {pkt[IP].dst}:{pkt[TCP].dport}"
    return None

def detect_null(pkt):
    if pkt.haslayer(TCP) and pkt[TCP].flags == 0:
        return f"[ALERT] TCP NULL Scan: {pkt[IP].src} → {pkt[IP].dst}:{pkt[TCP].dport}"
    return None

def detect_fin(pkt):
    if pkt.haslayer(TCP) and pkt[TCP].flags == "F":
        return f"[ALERT] TCP FIN Scan: {pkt[IP].src} → {pkt[IP].dst}:{pkt[TCP].dport}"
    return None
```

```
GNU nano 8.3
test_ids.py
import pytest
from scapy.all import IP, TCP, ICMP
from ids_core import detect_icmp, detect_syn, detect_null, detect_fin

def test_icmp_detection():
    pkt = IP(src="1.1.1.1", dst="2.2.2.2")/ICMP()
    alert = detect_icmp(pkt)
    assert alert is not None
    assert "ICMP Packet" in alert

def test_syn_detection():
    pkt = IP(src="1.1.1.1", dst="2.2.2.2")/TCP(flags="S", dport=80)
    alert = detect_syn(pkt)
    assert alert is not None
    assert "TCP SYN" in alert

def test_null_detection():
    pkt = IP(src="1.1.1.1", dst="2.2.2.2")/TCP(flags=0, dport=80)
    alert = detect_null(pkt)
    assert alert is not None
    assert "TCP NULL Scan" in alert

def test_fin_detection():
    pkt = IP(src="1.1.1.1", dst="2.2.2.2")/TCP(flags="F", dport=80)
    alert = detect_fin(pkt)
    assert alert is not None
    assert "TCP FIN Scan" in alert
```

```
(myenv)-(kali@kali)-[~]
$ nano ids_core.py

(myenv)-(kali@kali)-[~]
$ nano test_ids.py

(myenv)-(kali@kali)-[~]
$ pip install pytest

Collecting pytest
  Downloading pytest-8.4.1-py3-none-any.whl.metadata (7.7 kB)
Collecting iniconfig<1 (from pytest)
  Downloading iniconfig-2.1.0-py3-none-any.whl.metadata (2.7 kB)
Requirement already satisfied: packaging>=20 in ./myenv/lib/python3.13/site-packages (from pytest) (25.0)
Collecting pluggy<2, >=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl.metadata (4.8 kB)
Collecting pygments>=2.7.2 (from pytest)
  Downloading pygments-2.19.2-py3-none-any.whl.metadata (2.5 kB)
Downloading pytest-8.4.1-py3-none-any.whl (365 kB)
Downloading iniconfig-2.1.0-py3-none-any.whl (6.0 kB)
Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Downloading pygments-2.19.2-py3-none-any.whl (1.2 MB)
1.2/1.2 MB 1.1 MB/s eta 0:00:00
Installing collected packages: pygments, pluggy, iniconfig, pytest
Successfully installed iniconfig-2.1.0 pluggy-1.6.0 pygments-2.19.2 pytest-8.4.1

(myenv)-(kali@kali)-[~]
$ pytest test_ids.py -v

platform linux -- Python 3.13.2, pytest-8.4.1, pluggy-1.6.0 -- /home/kali/myenv/bin/python3
cachedir: .pytest_cache
rootdir: /home/kali
collected 4 items

test_ids.py::test_icmp_detection PASSED
test_ids.py::test_syn_detection PASSED
test_ids.py::test_null_detection PASSED
test_ids.py::test_fin_detection PASSED
```

3. False Positives and Next Steps

3.1 False-Positive Considerations

While the IDS was able to flag ICMP, SYN, NULL, and FIN traffic successfully, there are scenarios where legitimate traffic may trigger alerts:

- **ICMP Floods:** Heavy but legitimate network monitoring or connectivity checks may look like malicious ping floods.
- **SYN Flood Detection:** High connection attempts from load balancers or busy web servers may be mistaken for SYN scans.
- **Port Scans vs. Service Discovery:** Automated tools (e.g., vulnerability scanners used by admins) can be flagged as hostile scans.

To reduce false positives, the system could use adaptive thresholds, whitelisting for trusted hosts, or correlation with host-based logs.

3.2 Next-Step Ideas

To improve the IDS, the following enhancements are suggested:

1. **Anomaly Detection:** Introduce statistical or ML-based methods to detect unusual traffic patterns beyond simple thresholds.
2. **Signature Expansion:** Add detection for more protocols (e.g., UDP scans, DNS tunneling, HTTP anomalies).
3. **Performance Optimization:** Benchmark the IDS on high-throughput traffic to ensure scalability.
4. **Alert Management:** Integrate with a SIEM (Security Information and Event Management) system for centralized logging and analysis.
5. **Visualization:** Build a dashboard to show alerts, traffic stats, and historical patterns for easier monitoring.